

Retrieval-Augmented Generation (RAG)

Exam-Oriented Complete Overview

A compact revision guide covering the concepts most useful for university exams, interviews, and practical RAG development.

1. What is RAG?

RAG = Retrieval-Augmented Generation. It combines an information-retrieval system with a generative Large Language Model (LLM). Instead of relying only on knowledge stored in model parameters, the system retrieves relevant external information and supplies it to the LLM as context.

Core idea: Retrieve → Augment → Generate.

- Retrieve: find relevant information from a knowledge base.
- Augment: add the retrieved information to the user query/prompt.
- Generate: let the LLM produce an answer grounded in the supplied context.

RAG is useful for private data, frequently changing information, domain-specific knowledge, and applications where source grounding is important.

2. Why do we need RAG?

- LLMs can hallucinate or generate plausible but unsupported information.
- Pretrained model knowledge can become outdated.
- The model may not have access to private organizational documents.
- Fine-tuning is not always suitable for frequently changing knowledge.
- RAG allows external knowledge to be updated without retraining the entire LLM.

Exam point: RAG primarily changes the information available to the model at inference time; fine-tuning changes model parameters.

3. Basic RAG Architecture

Offline / Indexing phase:

Documents → Loading → Cleaning → Chunking → Embeddings → Vector Database

Online / Query phase:

User Query → Query Embedding → Retrieval → Optional Reranking → Context + Query → LLM → Answer

A production RAG system can additionally include connectors, metadata filtering, hybrid search, guardrails, access control, caching, observability, and evaluation.

4. Document Ingestion

Documents may come from PDFs, TXT files, DOCX files, HTML pages, databases, APIs, cloud storage, code repositories, or other enterprise sources.

Document Loader

A document loader extracts content and often metadata such as source filename, page number, document type, or URL.

Cleaning and preprocessing

- Remove unwanted formatting or duplicated text.

- Normalize whitespace where appropriate.
- Handle scanned documents with OCR when necessary.
- Preserve useful metadata because metadata can later support filtering and citations.

5. Chunking

Large documents are divided into smaller pieces called **chunks**. Retrieval usually works better when searchable units contain focused pieces of information.

Important parameters

- **Chunk size:** approximate amount of text/tokens in each chunk.
- **Chunk overlap:** repeated content between neighboring chunks to reduce loss of context at boundaries.

Common strategies

- Fixed-size chunking
- Recursive character/text splitting
- Sentence/paragraph-based chunking
- Semantic chunking
- Structure-aware chunking for headings, tables, code, etc.

Trade-off: chunks that are too small may lose context; chunks that are too large may reduce retrieval precision and consume more context window.

6. Embeddings

An embedding is a numerical vector representation of text that captures semantic information. Documents and queries are transformed into vectors so that semantically related content can be found using vector similarity.

Important rule: The embedding model used for indexing should be compatible with the embedding model used for query encoding.

Example

“How does machine learning work?” and “Explain the working of ML” should ideally have nearby embeddings even though their exact words differ.

7. Vector Database / Vector Store

A vector database stores embeddings and associated text/metadata and supports efficient similarity search.

- ChromaDB
- FAISS
- Pinecone
- Weaviate
- Qdrant
- Milvus

A stored record commonly contains: **ID + embedding + document/chunk text + metadata**.

8. Similarity Search

The query is embedded and compared against stored vectors. The system returns the most similar chunks.

Common similarity measures

- **Cosine similarity:** compares vector direction/angle; commonly used for semantic similarity.

- **Dot product:** computes the inner product of vectors; behavior depends on vector normalization.
- **Euclidean distance:** measures straight-line distance between vectors.

Top-K: If K=5, the retriever initially returns the five highest-ranked candidates.

9. Retrieval Methods

Dense / Vector Retrieval

Uses embeddings and semantic similarity. It is good when the query and document use different words but express similar meaning.

Sparse / Keyword Retrieval

Uses lexical matching. **BM25** is a classic sparse retrieval method. It is useful for exact terms, identifiers, names, error codes, and rare keywords.

Hybrid Retrieval

Combines dense semantic retrieval with sparse keyword retrieval. This can capture both semantic similarity and exact-term matching.

10. Reranking

Initial retrieval is often optimized for recall: retrieve a reasonably broad candidate set. A reranker then reorders those candidates using a stronger relevance model.

Example: retrieve top 20 → rerank → send best 5 to the LLM.

Why rerank? A vector similarity score alone may not perfectly represent query-document relevance. Reranking can improve precision among the retrieved candidates.

11. Query Transformation

The original user query may not be optimal for retrieval.

- **Query rewriting:** rewrite an ambiguous query into a retrieval-friendly form.
- **Query expansion:** add useful terms or concepts.
- **Multi-query retrieval:** generate multiple related queries and combine their results.
- **HyDE:** generate a hypothetical answer/document and use it to improve retrieval in some setups.

These methods belong to the broader family of retrieval optimization techniques.

12. Augmentation / Prompt Construction

Retrieved chunks are placed into a prompt along with the original user question.

Typical structure:

System instructions + Retrieved context + User question → LLM

A good prompt should tell the model how to use the context and what to do when the answer is not supported by the retrieved information.

Grounding instruction example: Answer using the provided context; if the context does not contain the answer, state that the information is unavailable rather than inventing it.

13. Generation

The LLM receives the query and retrieved context and generates the final response. OpenAI, Mistral, Anthropic, Google, and open-source models can all be used as generators depending on the application.

In a multi-model RAG project, the same retrieval pipeline can feed different LLMs, allowing comparison of answer quality, latency, cost, and faithfulness.

14. Context Window

The context window is the amount of input/output token information a model can process within its supported limit. More retrieved context is not automatically better.

- Too little context → relevant evidence may be missing.
- Too much context → irrelevant information, higher cost/latency, and possible distraction.

The objective is to provide **relevant, sufficient, and well-organized context**.

15. Metadata Filtering

Metadata can restrict retrieval before or during vector search.

Example metadata: department=HR, year=2026, document_type=policy, page=12.

Filtering is particularly useful for multi-tenant systems, access control, dates, document types, and domain-specific search.

16. Grounding and Hallucination

RAG can reduce hallucinations by supplying evidence, but it does **not** guarantee factual correctness.

- Bad retrieval → bad context → potentially bad answer.
- Good retrieval → better evidence, but the LLM can still misinterpret or ignore it.

Key exam sentence: RAG reduces dependence on parametric memory by grounding generation in retrieved external context.

17. RAG vs Fine-Tuning vs Long Context

Technique	Main purpose	Knowledge update	Typical use
RAG	Retrieve external knowledge at inference	Easy	Private/current knowledge
Fine-tuning	Change model behavior/parameters	Requires retraining	Style, behavior, domain patterns
Long context	Provide a large input directly	Easy per request	Small number of large documents

18. RAG Evaluation

RAG must be evaluated at both the **retrieval** and **generation** levels.

Retrieval metrics

- **Precision@K:** fraction of retrieved top-K items that are relevant.
- **Recall@K:** fraction of relevant items successfully retrieved in top-K.
- **MRR:** rewards systems that place the first relevant result high in the ranking.
- **MAP:** summarizes average precision across relevant ranks.
- **NDCG:** evaluates ranking quality while giving higher importance to higher positions.

Generation / RAG metrics

- **Faithfulness:** whether the answer is supported by the retrieved context.
- **Answer relevance:** whether the response addresses the question.
- **Context relevance:** whether retrieved context is relevant to the query.
- **Context recall:** whether needed information was successfully retrieved.

RAGAS is a commonly used framework for evaluating RAG pipelines.

19. Common RAG Failure Modes

- **Bad chunking:** important information is split incorrectly or context is lost.
- **Embedding mismatch:** poor semantic representation leads to poor retrieval.
- **Low recall:** the required chunk is not retrieved.
- **Low precision:** too many irrelevant chunks are retrieved.
- **Keyword failure:** semantic search misses exact identifiers or terms.
- **Context overload:** too much irrelevant context is passed to the LLM.
- **Hallucination:** model produces unsupported information.
- **Stale index:** vector store has not been updated after source documents change.
- **Access-control failure:** a user retrieves documents they should not see.

20. Advanced RAG

- **Naive RAG:** basic chunk → embedding → vector search → prompt → LLM pipeline.
- **Advanced RAG:** adds query transformation, hybrid search, reranking, filtering, compression, better chunking, etc.
- **Modular RAG:** retrieval/generation modules can be composed and replaced independently.
- **Agentic RAG:** an agent can plan, choose tools/retrievers, perform multiple retrieval steps, and iterate.
- **Graph RAG:** uses graph structures/relationships to improve retrieval and multi-hop reasoning for suitable datasets.
- **Multimodal RAG:** retrieves information across text, images, tables, audio, or video when supported by the system.
- **Hybrid RAG:** combines multiple retrieval strategies, commonly dense + sparse.

21. Security and Production Concepts

- Authentication and authorization
- Tenant/user-level document isolation
- Prompt injection defense
- Sensitive-data handling
- Input/output validation
- Source citations and provenance
- Logging and observability
- Caching
- Rate limiting
- Index refresh/update strategy
- Latency and cost monitoring

22. Important RAG Terminology — One-Line Revision

Term	Meaning
RAG	Retrieval-Augmented Generation.
Chunk	A smaller searchable unit of a document.
Embedding	Numerical vector representation of information.
Vector Store	System that stores vectors and supports similarity search.

Term	Meaning
Retriever	Component that finds relevant information.
Top-K	Number of highest-ranked candidates returned.
BM25	Sparse lexical retrieval algorithm.
Hybrid Search	Combination of sparse and dense retrieval.
Reranker	Model/component that reorders retrieved candidates.
Context	Retrieved information supplied to the LLM.
Grounding	Constraining generation using supporting evidence.
Hallucination	Unsupported or incorrect generated content.
Metadata	Additional attributes attached to documents/chunks.
Faithfulness	Degree to which the answer is supported by context.
Recall@K	How much relevant information appears in top-K.
Precision@K	How much of top-K retrieved information is relevant.

23. Very Important Exam Questions

1. Define RAG. Explain its architecture and working.
2. Why is RAG preferred over relying only on an LLM's pretrained knowledge?
3. Explain document chunking and the effect of chunk size and overlap.
4. What are embeddings? Why are they required in vector retrieval?
5. Explain how a vector database works in a RAG system.
6. Compare cosine similarity, dot product, and Euclidean distance.
7. What is Top-K retrieval?
8. Compare dense retrieval and sparse retrieval.
9. What is BM25? Why can it complement vector search?
10. What is hybrid retrieval?
11. What is reranking and why is it useful?
12. Explain query rewriting and multi-query retrieval.
13. What is hallucination? How can RAG reduce it?
14. Compare RAG and fine-tuning.
15. Explain precision@K, recall@K, MRR, and NDCG.
16. What is faithfulness in RAG evaluation?
17. Explain naive RAG, advanced RAG, and agentic RAG.
18. What is metadata filtering and why is it important?
19. Explain the difference between retrieval quality and generation quality.
20. What are common failure modes of a RAG system?

24. 10-Mark Answer Template

For a long-answer question such as "Explain RAG architecture," use this order:

- Definition of RAG

- Problem addressed: hallucination, outdated/private knowledge
- Architecture diagram
- Indexing phase: loading → preprocessing → chunking → embeddings → vector store
- Query phase: query → retrieval → reranking → context → LLM
- Explain dense/sparse/hybrid retrieval
- Explain generation and grounding
- Advantages
- Limitations
- Applications
- Evaluation metrics

Conclusion: RAG connects LLMs to external knowledge and can improve relevance, freshness, and grounding without requiring the knowledge itself to be fully stored in the model parameters.

25. Quick Memory Trick

INDEX → RETRIEVE → AUGMENT → GENERATE → EVALUATE

Index: prepare and store knowledge. Retrieve: find relevant chunks. Augment: put evidence into the prompt. Generate: produce the answer. Evaluate: measure retrieval and generation quality.

26. References

AWS Prescriptive Guidance, "Understanding Retrieval Augmented Generation."

Google Cloud, "What is Retrieval-Augmented Generation (RAG)?"

Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," 2023.

Yu et al., "Evaluation of Retrieval-Augmented Generation: A Survey," 2024.

These references were used to verify the core RAG architecture, retrieval/generation stages, and evaluation concepts.